



KSFUPRO1K1U – Functional Programming

Lecture 14: ReCap

Niels Hallenberg

These slides are based on original slides by Michael R. Hansen, DTU. Thanks!!!



The original slides has been used at a course in functional programming at DTU.

Recursion. Example $n! = 1 \cdot 2 \cdot \dots \cdot n$, $n \geq 0$



Mathematical definition:

recursion formula

$$\begin{aligned} 0! &= 1 & (i) \\ n! &= n \cdot (n-1)!, \quad \text{for } n > 0 & (ii) \end{aligned}$$

Computation:

$$\begin{aligned} &3! \\ = &3 \cdot (3-1)! & (ii) \\ = &3 \cdot 2 \cdot (2-1)! & (ii) \\ = &3 \cdot 2 \cdot 1 \cdot (1-1)! & (ii) \\ = &3 \cdot 2 \cdot 1 \cdot 1 & (i) \\ = &6 \end{aligned}$$



Function declaration:

```
let rec fact = function
  | 0 -> 1                (* i *)
  | n -> n * fact(n-1);;  (* ii *)
val fact : int -> int
```

Evaluation:

```
fact(3)
~> 3 * fact(3 - 1)      (ii) [n ↦ 3]
~> 3 * 2 * fact(2 - 1)  (ii) [n ↦ 2]
~> 3 * 2 * 1 * fact(1 - 1) (ii) [n ↦ 1]
~> 3 * 2 * 1 * 1        (i)  [n ↦ 0]
~> 6
```

$e_1 \rightsquigarrow e_2$ reads: e_1 evaluates to e_2

Recursion. Example $x^n = x \cdot \dots \cdot x$, n occurrences of x



Mathematical definition:

recursion formula

$$x^0 = 1 \quad (1)$$

$$x^n = x \cdot x^{n-1}, \quad \text{for } n > 0 \quad (2)$$

Function declaration:

```
let rec power = function
| (_, 0) -> 1.0                (* 1 *)
| (x, n) -> x * power (x, n-1) (* 2 *)
```

Patterns:

$(_, 0)$ matches any **pair** of the form $(x, 0)$.

The **wildcard** pattern $_$ matches any value.

(x, n) matches any pair (u, i) **yielding** the bindings

$$x \mapsto u, n \mapsto i$$

Evaluation. Example: `power(4.0, 2)`



Function declaration:

```
let rec power = function
  | (_, 0) -> 1.0                (* 1 *)
  | (x, n) -> x * power(x, n-1)  (* 2 *)
```

Evaluation:

<code>power(4.0, 2)</code>	
$\rightsquigarrow$ <code>4.0 * power(4.0, 2 - 1)</code>	Clause 2, $[x \mapsto 4.0, n \mapsto 2]$
$\rightsquigarrow$ <code>4.0 * power(4.0, 1)</code>	
$\rightsquigarrow$ <code>4.0 * (4.0 * power(4.0, 1 - 1))</code>	Clause 2, $[x \mapsto 4.0, n \mapsto 1]$
$\rightsquigarrow$ <code>4.0 * (4.0 * power(4.0, 0))</code>	
$\rightsquigarrow$ <code>4.0 * (4.0 * 1)</code>	Clause 1
$\rightsquigarrow$ <code>16.0</code>	

Function composition: $(f \circ g)(x) = f(g(x))$



For example, if $f(y) = y + 3$ and $g(x) = x^2$, then $(f \circ g)(z) = z^2 + 3$.

The infix operator `<<` in F# denotes functional composition:

```
let f y = y+3;;           // f(y) = y+3

let g x = x*x;;          // g(x) = x*x

let h = f << g;;          // h = (f o g)
val h : int -> int

h 4;;                     // h(4) = (f o g) (4)
val it : int = 19
```

Using just anonymous functions:

```
((fun y -> y+3) << (fun x -> x*x)) 4;;
val it : int = 19
```

Type of `(<<)` ?



The operator $|>$ means “*send the value as argument to the function at the right*”

arg $|>$ *fct* is equivalent to *fct arg*

The operator $<|$ means “*send the value as argument to the functions at the left*”

fct $|<$ *arg* is equivalent to *fct arg*

For example $a+b |> \sin$ and $\sin <| a+b$ means $\sin(a+b)$.

The two functions are predefined:

```
let (.<|. ) f a = f a
let (.|>. ) a f = f a
```

Type of $(|>)$?



It is often useful to have **when guards** in patterns:

```
let ordText x y = match compare x y with
    | t when t > 0 -> "greater"
    | 0            -> "equal"
    | _           -> "less";;

ordText "abc" "Abc";;
val it : string = "greater"
```

The first clause is only taken when `t > 0` evaluates to true.



Extract components of tuples

```
let ((x,_), (_,y,_)) = ((1,true), ("a", "b", false));;  
val x : int = 1  
val y : string = "b"
```

Pattern matching yields bindings

Restriction

```
let (x,x) = (1,1);;  
...  
... ERROR ... 'x' is bound twice in this pattern
```

Will below give you a warning?

```
let xs = [1;2;3];;  
let [x;y;z] = xs;;
```

What about

```
let y::ys = xs;;
```



Example: sum of norms of geometric vectors:

```
let norm(x1:float,y1:float) = sqrt(x1*x1+y1*y1);;  
val norm : float * float -> float
```

```
let rec sumOfNorms = function  
  | []      -> 0.0  
  | v::vs -> norm v + sumOfNorms vs;;  
val sumOfNorms : (float * float) list -> float
```

```
let vs = [(1.0,2.0); (2.0,1.0); (2.0, 5.5)];;  
val vs : (float * float) list  
        = [(1.0, 2.0); (2.0, 1.0); (2.0, 5.5)]
```

```
sumOfNorms vs;;  
val it : float = 10.32448591
```



```
let rec sumOfNorms = function
  | []      -> 0.0
  | v::vs   -> norm v + sumOfNorms vs;;
```

Let $f\ v\ s$ abbreviate $\text{norm } v + s$ in the evaluation:

$$\begin{aligned} & \text{sumOfNorms } [v_0; v_1; \dots; v_{n-1}] \\ \rightsquigarrow & \text{norm } v_0 + (\text{sumOfNorms } [v_1; \dots; v_{n-1}]) \\ = & f\ v_0 (\text{sumOfNorms } [v_1; \dots; v_{n-1}]) \\ \rightsquigarrow & f\ v_0 (f\ v_1 (\text{sumOfNorms } [v_2; \dots; v_{n-1}])) \\ & \vdots \\ \rightsquigarrow & f\ v_0 (f\ v_1 (\dots (f\ v_{n-1} 0.0) \dots)) \end{aligned}$$

This repeated application of f is also called a **folding** of f .

Many functions follow such recursion and evaluation schemes

Higher-order list functions: `foldBack` (1)



Suppose that $\otimes$ is an infix function. Then

$$\begin{aligned}\text{foldBack } (\otimes) \ [a_0; a_1; \dots; a_{n-2}; a_{n-1}] \ e_b \\ = \ a_0 \otimes (a_1 \otimes (\dots (a_{n-2} \otimes (a_{n-1} \otimes e_b)) \dots))\end{aligned}$$

$$\begin{aligned}\text{List.foldBack } (+) \ [1; 2; 3] \ 0 &= 1 + (2 + (3 + 0)) = 6 \\ \text{List.foldBack } (-) \ [1; 2; 3] \ 0 &= 1 - (2 - (3 - 0)) = 2\end{aligned}$$

Using the cons operator gives the append function `@` on lists:

$$\begin{aligned}\text{foldBack } (\text{fun } x \text{ rst} \rightarrow x::\text{rst}) \ [x_0; x_1; \dots; x_{n-1}] \ ys \\ = x_0::(x_1:: \dots ::(x_{n-1}::ys) \dots) \\ = [x_0; x_1; \dots; x_{n-1}] \ @ \ ys\end{aligned}$$

so we get:

```
let (@) xs ys = List.foldBack (fun x rst -> x::rst) xs ys;;
val ( @ ) : 'a list -> 'a list -> 'a list

[1;2] @ [3;4] ;;
val it : int list = [1; 2; 3; 4]
```

Declaration of `foldBack`



```
let rec foldBack f xlst e =  
  match xlst with  
  | x::xs -> f x (foldBack f xs e)  
  | []      -> e ;;  
val foldBack : ('a -> 'b -> 'b) -> 'a list -> 'b -> 'b
```

```
let sumOfNorms vs = foldBack (fun v s -> norm v + s) vs 0.0;;
```

```
let length xs = foldBack (fun _ n -> n+1) xs 0;;
```

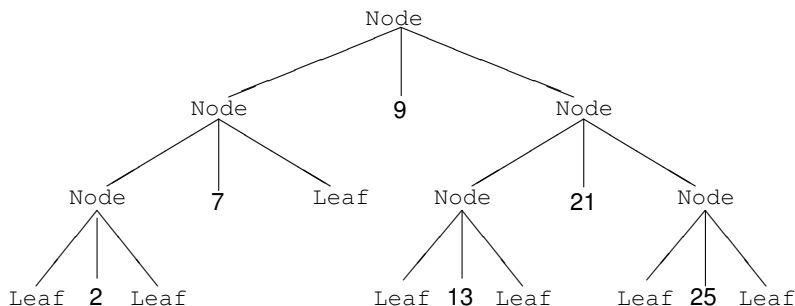
```
let map f xs = foldBack (fun x rs -> f x :: rs) xs [];;
```

Does `foldBack` always run through the entire list, assuming `f` returns a value?



A *finite tree* is a value which may contain a subcomponent of the same type.

Example: A *binary search tree*



Condition: for every node containing the value x : every value in the left subtree is smaller than x , and every value in the right subtree is greater than x .

Example: Binary Trees



A *recursive datatype* is used to represent values which are trees.

```
type Tree = Leaf
          | Node of Tree*int*Tree;;

Leaf;;
val it : Tree = Leaf

Node;;
val it : Tree * int * Tree -> Tree = <fun:clo@4>
```

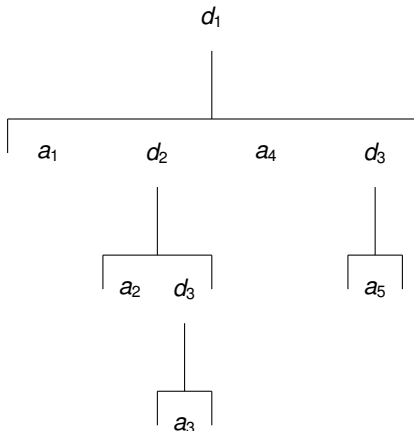
The two parts in the declaration are **rules** for generating trees:

- `Leaf` is a tree
- if t_1, t_2 are trees, n is an integer, then `Node( $t_1, n, t_2$ )` is a tree.

The tree from the previous slide is denoted by:

```
Node (Node (Node (Leaf, 2, Leaf) , 7, Leaf) ,
      9,
      Node (Node (Leaf, 13, Leaf) , 21, Node (Leaf, 25, Leaf) ) )
```

Mutual recursion. Example: File system



- A **file system** is a list of **elements**
- an **element** is a file or a directory, which is a named **file system**



- are combined using **and**

```
type FileSys = Element list
and Element =
  | File of string
  | Dir of string * FileSys
```

```
let d1 =
  Dir("d1", [File "a1";
              Dir("d2", [File "a2";
                          Dir("d3", [File "a3"])]);
              File "a4";
              Dir("d3", [File "a5"])
  ])
```

The type of d1 is ?



- are combined using **and**

Example: extract the names occurring in file systems and elements.

```
let rec namesFileSys = function
  | []      -> []
  | e::es -> (namesElement e) @ (namesFileSys es)
and namesElement = function
  | File s      -> [s]
  | Dir(s,fs) -> s :: (namesFileSys fs) ;;
val namesFileSys : Element list -> string list
val namesElement : Element -> string list

namesElement d1 ;;
val it : string list = ["d1"; "a1"; "d2"; "a2";
                        "d3"; "a3"; "a4"; "d3"; "a5"]
```



You can also have a mutable record field

```
> type intRec = {mutable count : int } ;;
```

```
type intRec =  
  {mutable count: int;}
```

```
> let r1 = {count = 0} ;;
```

```
val r1 : intRec = {count = 0;}
```

```
> let incr x =  
  x.count <- x.count + 1  
  x.count ;;
```

```
val incr : intRec -> int
```

```
> incr r1;;  
val it : int = 1  
> incr r1;;  
val it : int = 2
```

Environment

$[r1 \mapsto \{count \mapsto h_1\}]$

Store

$[h_1 \mapsto 2]$



Say we have a mutable record field and a few operators:

```
type 'a ref = {mutable contents: 'a}
let ref v = {contents = v}
let (!) r = r.contents
let (:=) r v = r.contents <- v
```

Then execute the following:

```
let a = ref 5
> val a : int ref = {contents = 5;}
let b = a
> val b : int ref = {contents = 5;}
```

A record is a reference type and hence heap allocated

Environment

$$\left[\begin{array}{l} a \mapsto \{\text{contents} \rightarrow l_1\} \\ b \mapsto \{\text{contents} \rightarrow l_1\} \end{array} \right]$$

Store

$$[l_1 \mapsto 5]$$

The two variables `a` and `b` are stack allocated and both pointing at the same record stored in the heap.

This is how references `ref`'s are implemented in F#.



One may ask why we have both `mutable` and `ref`. Other functional languages, e.g., Standard ML only have references.

- The compiler automatically inserts the `contentsOf` operator when you use `mutable`. You have explicitly to dereference a reference with `!`.
- A mutable value is stack allocated. A reference (`ref`) is heap allocated, as it is implemented as a record.
- You can alias two references. You can't do that with mutables (continuing from previous slide):

```
b := 10
> val it : unit = ()
!a
> val it : int = 10
!b
> val it : int = 10

let mutable a' = 5
> val mutable a' : int = 5

let mutable b' = a'
> val mutable b' : int = 5
b' <- 10
> val it : unit = ()
a'
> val it : int = 5
b'
> val it : int = 10
```

- Due to restrictions on polymorphic expressions, Section 4.5, you can't declare the following top level mutable: `let mutable a = [];;`.



Efficient solutions are obtained by using *more general functions*:

$$\begin{aligned}\text{factA}(n, m) &= n! \cdot m, \text{ for } n \geq 0 \\ \text{revA}([x_1, \dots, x_n], ys) &= [x_n, \dots, x_1] @ ys\end{aligned}$$

We have:

$$\begin{aligned}n! &= \text{factA}(n, 1) \\ \text{rev } [x_1, \dots, x_n] &= \text{revA}([x_1, \dots, x_n], [])\end{aligned}$$

m and *ys* are called *accumulating parameters*. They are used to hold the temporary result during the evaluation.



Accumulating parameters are not sufficient to achieve a tail-recursive version for arbitrary recursive functions.

Consider for example:

```
type BinTree<'a> =  
  | Leaf  
  | Node of BinTree<'a> * 'a * BinTree<'a>;;  
  
let rec count = function  
  | Leaf          -> 0  
  | Node(tl,n,tr) -> count tl + count tr + 1;;
```

A counting function:

```
countA: int -> BinTree<'a> -> int
```

using an accumulating parameter will **not be tail-recursive** due to the expression containing recursive calls on the left and right sub-trees.
(Ex. 9.8)

Example: Tail-recursive count



```
let rec countC t c =  
  match t with  
  | Leaf          -> c 0  
  | Node(tl,n,tr) ->  
    countC tl (fun vl -> countC tr (fun vr -> c(vl+vr+1)))  
val countC : BinTree<'a> -> (int -> 'b) -> 'b  
  
countC (Node(Node(Leaf,1,Leaf),2,Node(Leaf,3,Leaf))) id;  
val it : int = 3
```

- Both calls of `countC` are tail calls
- The calls of the `c` is tail call

Hence, the stack will not grow when evaluating `countC t c`.

- `countC` can handle bigger trees than `count`
- `count` is faster



Sequence expressions can be used for defining step-by-step generation of sequences.

Remove multiples of *a* from sequence *sq*:

```
let sift a sq = Seq.filter (fun n -> n % a <> 0) sq;;  
val sift : int -> seq<int> -> seq<int>
```

The sieve of Eratosthenes:

```
let rec sieve sq =  
    seq { let p = Seq.item 0 sq  
          yield p  
          yield! sieve(sift p (Seq.skip 1 sq)) };;  
val sieve : seq<int> -> seq<int>
```

- By construction lazy – no explicit `Seq.delay` is needed
- `yield x` adds the element *x* to the generated sequence
- `yield! sq` adds the sequence *sq* to the generated sequence
- `seqexp1`
`seqexp2` appends the sequence of `seqexp1` to that of `seqexp2`



Source: [https:](https://msdn.microsoft.com/en-us/library/dd233248.aspx)

[//msdn.microsoft.com/en-us/library/dd233248.aspx](https://msdn.microsoft.com/en-us/library/dd233248.aspx)

Active patterns makes it possible to decompose data into customized partitions. Data is subdivided into partitions which you name. These names can be used in pattern matching.

```
let (|Even|Odd|) input =  
    if input % 2 = 0 then Even else Odd  
  
let TestNumber input =  
    match input with  
    | Even -> printfn "%d is even" input  
    | Odd -> printfn "%d is odd" input  
  
TestNumber 7  
TestNumber 11  
TestNumber 32
```



Source: <http://fsharpforfunandprofit.com/posts/convenience-active-patterns/>

Active patterns that do not always produce a value are called *partial active patterns*; they have a return value that is an option type.

```
let (|Int|_|) (str:string) =  
    match System.Int32.TryParse(str) with  
    | (true,i) -> Some i  
    | _ -> None
```

```
let (|Bool|_|) (str:string) =  
    match System.Boolean.TryParse(str) with  
    | (true,b) -> Some b  
    | _ -> None
```



```
let testParse str =  
  match str with  
  | Int i -> printfn "The value is an int '%i'" i  
  | Bool b -> printfn "The value is a bool '%b'" b  
  | _ -> printfn "The value '%s' is something else" str
```

```
testParse "12"  
testParse "true"  
testParse "abc"
```

```
> The value is an int '12'  
val it : unit = ()  
> The value is a bool 'true'  
val it : unit = ()  
> The value 'abc' is something else  
val it : unit = ()
```



Advice for exam preparation

- Go through the book and slides.
- Go through all your homework
- Practice on old exam sets.

The Exam

- F# language constructs
- Small functions, (recursive of course)
- Datatypes and manipulating data
- Data structures like Lists, Sets, Maps.
- Sequences
- Tail recursiveness, accumulating parameter, continuations.
- Problem solving

There are plenty of exam sets and thereby a lot to practice on.
Please use Discussion forum for questions and discussions.